Project Report On

Predictive Weather Guidance With Route Engine

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

NIMISH M BIJU

41523404089

(SIXTH SEMESTER)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEETING
UNIVERSITY COLLEGE OF ENGINEERING, KARIAVATTOM
THIRUVANANTHAPURAM APRIL 2026

CERTIFICATE

This is to certify that the seminar report entitled "**Predictive Weather Guidance With Route Engine**"

Is a Bonafide record of the Seminar presented in the sixth semester by

NIMISH M BIJU

41523404089

In partial fulfillment for the award of Degree of Bachelor of Technology in

Computer Science and Engineering

From the university of kerala for the academic year 2023-2027

Project Supervisor	Project Coordinator	Head of Department
SURYA KRISHNAN G R	GARGY G	DIVYA DT

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to Prof. Saina A R, Principal, University College of Engineering, Kariyavattom , for the facilities she has provided in the college without which the project could not have been completed.

I acknowledge my sincere gratitude to Ms. Divya D.T., Head of the Department, Department of Computer Science and Engineering, University College of Engineering, Kariyavattom, for her support and Encouragement.

I am greatly indebted to my Seminar Guide Ms. Surya Krishnan G R, Assistant Professor, Department of Computer Science and Engineering, University College of Engineering, Kariyavattom for her valuable guidance and Encouragement.

I also acknowledge my gratitude to all other members of all faculty of the Department of Computer Science and Engineering.

We also express our wholehearted gratefulness to the College Management and to all those who helped me make this undertaking a success.

Above all, we thank our Parents and God Almighty without whose blessings this seminar would not have been a reality.

Last but not least, I am also thankful to all my Friends who gave me valuable suggestions and helped me completion of this seminar work.

Thank you

ABSTRACT

Traditional navigation systems calculate the fastest route between two points but frequently fail to account for dynamic environmental factors, such as changing weather conditions over the duration of a journey. This project proposes and implements a **Predictive Weather Guidance and Route Planning System**, a web-based application designed to integrate meteorological forecasting directly into the pathfinding process. Built entirely as a client-side application, the system utilizes an asynchronous architecture combining the OpenStreetMap Nominatim API for geocoding, Leaflet Routing Machine (OSRM) for trajectory mapping and travel time estimation, and the OpenWeather API for fetching temporal meteorological data. Unlike standard applications that display static current weather, this system mathematically predicts the user's exact arrival time at critical waypoints (Start, Midpoint, and Destination) based on user-defined departure times and driving distance. It then queries a 5-day predictive weather model to extract the specific 3-hour forecast block corresponding to those future arrival times. Furthermore, the system includes an automated hazard-detection algorithm that analyzes the fetched weather codes to generate real-time safety alerts for rain, strong winds, fog, and thunderstorms. By successfully bridging spatial navigation with predictive meteorology in a seamless "glassmorphism" user interface, this project demonstrates a highly practical approach to Advanced Traveler Information Systems (ATIS), ultimately enhancing driver awareness, route planning efficiency, and overall road safety.

In the documented full implementation, the route-prediction interface is further supported by a lightweight backend layer built with Node.js and Express.js. This backend preserves requested-route history in a protected data store, provides an authenticated administrative portal, and introduces CAPTCHA-based human verification together with bcrypt-based password hashing for secure login. The result is a more complete full-stack system in which predictive weather guidance, route-history persistence, and controlled administrative access operate as a single integrated web application.

TABLE OF CONTENTS

TABLE OF CONTENTS

ABSTRACT 4

CHAPTER 1 INTRODUCTION 9

1.1 Background 9

1.2 Problem Statement 9

1.3 Objectives 10

1.4 Scope and Significance 10

CHAPTER 2 LITERATURE REVIEW..... 12

CHAPTER 3 SYSTEM ANALYSIS AND PROPOSED DESIGN 13

3.1 Proposed System 13

3.2 Overall Architecture 14

3.3 Functional and Non-Functional Requirements 14

3.4 Feasibility Analysis 15

3.5 Existing Fragmented Workflow 15

CHAPTER 4 ROAD MAPPING AND PATH COMPUTATION ALGORITHM 19

4.1 Algorithm Description 16

4.2 Technologies Used 16

4.3 Implementation Details 17

4.4 Pseudocode And Architecture 18

CHAPTER 5 WEATHER FORECAST EXTRACTION AND API INTEGRATION ALGORITHM ... 19

5.1 Algorithm Description 19

5.2 Technologies Used 19

5.3 Implementation Details 20

5.4 Pseudocode And Architecture 21

CHAPTER 6 ROUTE–WEATHER SYNCHRONISATION AND TIMELINE GENERATION ALGORITHM ... 22

6.1 Algorithm Description 22

6.2 Technologies Used 22

6.3 Implementation Details 22

6.4 Pseudocode And Architecture 23

CHAPTER 7 WARNING GENERATION ALGORITHM 24

7.1 Algorithm Description 24

7.2 Technologies Used 24

7.3 Implementation Details 24

7.4 Pseudocode And Architecture 25

CHAPTER 8 FRONTEND ORCHESTRATION ALGORITHM 26

8.1 Algorithm Description 26

8.2 Technologies Used 26

8.3 Implementation Details 26

8.4 Pseudocode And Architecture 27

CHAPTER 9 OUTPUT, DISCUSSION, ADVANTAGES AND LIMITATIONS ... 28

9.1 Output and Discussion 28

9.2 Advantages 28

9.3 Limitations 29

CHAPTER 10 CONCLUSION AND FUTURE SCOPE 30

10.1 Conclusion 30

10.2 Future Scope 30

CHAPTER 11 REFERENCES 32

CHAPTER 12 PUBLICATIONS 33

LIST OF TABLES

Table 1: Comparison of Existing Fragmented Workflow and Proposed System 16

Table 2: External Services Used by the Application 20

Table 3: Route-Point and Forecast-Time Matching Concept 23

Table 4: Technical Stack and Implementation Role 2

LIST OF FIGURES

Fig 1: ROAD MAPPING AND PATH COMPUTATION ALGORITHM 18

Fig 2:WEATHER FORECAST EXTRACTION AND API INTEGRATION ALGORITHM 21

Fig 3: ROUTE–WEATHER SYNCHRONISATION AND TIMELINE GENERATION ALGORITHM..... 23

Fig 4: WARNING GENERATION ALGORITHM 25

Fig 5:FRONTEND ORCHESTRATION ALGORITHM..... 27

CHAPTER 1

INTRODUCTION

1.1 Background

Digital navigation has become a routine component of commuting, tourism, logistics, and inter-city travel. In its simplest form, a route planner receives an origin and a destination and returns a path through the road network, normally optimized for shortest distance or minimum estimated travel time. Although this form of optimization is useful, it does not fully represent the realities of travel. Real-world journeys are affected not only by road geometry and distance, but also by weather, visibility, atmospheric hazards, and the expected time at which each route segment is reached.

Conventional travel planning is therefore fragmented. Users often consult one system for navigation and another system for weather. A route application tells them how long the trip may take, while a weather application tells them what the forecast looks like for one or more places. The user must mentally combine these outputs and infer whether the route is still suitable under forecasted conditions. That manual interpretation becomes difficult when the departure is planned for later in the day, when forecast panels are organized into discrete time blocks, or when different parts of the route traverse different localities.

The present project, Predictive Weather Guidance with Route Planning, addresses this gap by coupling route geometry with route timing and then coupling that timing with forecast data. The result is not merely a route planner with a weather panel and not merely a weather app with a map. It is a route-aware forecasting interface that attempts to estimate what weather will be encountered when the traveller reaches each important point of the journey.

1.2 Problem Statement

- Existing route planning systems usually optimise for distance or travel time, but they do not explicitly align forecasted weather with the user's planned travel timeline.
- Users typically need to open multiple applications and manually compare route duration with forecast panels, which increases cognitive burden and may lead to poor judgement.
- Weather conditions vary along a route; conditions at the origin, intermediate segment, and destination may differ considerably, especially for longer or geographically diverse trips.

- A browser-based weather-aware route planner is needed to provide route-specific forecast guidance and warning messages in a single interface using live geospatial and forecast services.

1.3 Objectives

- To design a single-page web application that accepts a source, destination, and departure time from the user.
- To geocode human-readable place names into latitude–longitude coordinates and support reverse geocoding for route-derived points.
- To compute a drivable road route, display it on an interactive map, and extract route summary information such as total distance and estimated travel duration.
- To determine key route points, specifically the route start, the midpoint derived from the route polyline, and the destination.
- To calculate estimated arrival times for those points by combining departure time with route duration.
- To fetch weather forecast data for each route point and match the route timeline to the closest forecast timestamps.
- To generate contextual weather-risk warnings for conditions such as rain, thunderstorm, low visibility, and strong wind.
- To implement the full system in a frontend-only architecture suitable for educational deployment and mini-project demonstration.

1.4 Scope and Significance

The scope of the present project is deliberately focused. It does not attempt to build a commercial-grade navigation platform, nor does it claim to perform full route optimization under continuously changing weather fields. Instead, it demonstrates a clear and educationally valuable pipeline: place names are converted to coordinates, coordinates are converted to a route, the route is interpreted in time, and time-indexed forecast records are matched to route points.

The significance of the project lies in decision support. A traveller planning a trip from Kochi to Thrissur, for example, may want to know whether rain is likely during departure, whether the midpoint region may face thunderstorms, or whether strong wind is expected near the destination. The application automates that reasoning. This makes the system more

informative than a destination-only weather check and more context-aware than a route-only map view.

CHAPTER 2

LITERATURE REVIEW

Study / Reference	Model(s) Used	Data Type	Technique Type	Key Contribution
Anwar (2025) – Weather-aware route planner	Rule-based + API Integration (Google Maps + Weather API)	Real-time weather data + Geolocation	Web-based System / Data Integration	Integrates live weather data with navigation to improve safety and efficiency
Setiawan et al. (2023) – Travel Route Recommendation System	Machine Learning Models	Weather prediction + Geolocation data	Machine Learning	Predicts weather and suggests optimal routes
Ye et al. (2023) – Route Selection with Real-time Weather Data	Spatial Decision Support System (SDSS)	Real-time weather + GIS data	Data Analytics / GIS Integration	Improves route accuracy using real-time weather integration

CHAPTER 3

SYSTEM ANALYSIS AND PROPOSED DESIGN

3.1 Proposed System

The proposed system integrates user input capture, autocomplete, geocoding, route generation, midpoint extraction, reverse geocoding, timeline generation, forecast retrieval, nearest-time forecast matching, warning generation, and result rendering in a single-page browser application. The user provides the route and departure information once, and the application completes the remaining workflow automatically.

The HTML, CSS, and JavaScript confirm this design. The HTML declares the structural placeholders for map, inputs, summary, warnings, and weather cards. The CSS defines the visual and responsive behavior. The JavaScript performs the operational orchestration that binds all system components together.

Aspect	Existing fragmented workflow	Proposed application
User interaction	Route and weather checked separately	Single integrated planning workflow
Time matching	Estimated manually by the user	Computed automatically from route duration
Intermediate weather insight	Usually absent	Midpoint derived and analysed
Safety interpretation	Left to user judgement	Automatic warning banners generated
Hosting model	Varies by service	Static frontend deployment possible

Table 1. Comparison of existing fragmented workflow and proposed system

3.2 Overall Architecture

The application can be understood as a layered architecture moving from user input, through geocoding and routing, into route processing and timeline generation, and finally into forecast retrieval, warning generation, and presentation. This layered view is useful because it separates concerns: location interpretation, route geometry, temporal reasoning, environmental interpretation, and user communication are different technical problems even though they are executed in one browser workflow.

3.3 Functional and Non-Functional Requirements

Functional requirements include route generation, forecast retrieval, time-based forecast matching, and warning generation. Non-functional requirements include responsiveness, clarity of interaction, reasonable browser-side performance, lightweight deployment, and a visually coherent interface. The project overview also emphasizes cost effectiveness, privacy-respecting behavior, and ease of hosting because no persistent backend is used.

Functional Requirements

- To design a single-page web application that accepts a source, destination, and departure time from the user.
- To geocode human-readable place names into latitude–longitude coordinates and support reverse geocoding for route-derived points.
- To compute a drivable road route, display it on an interactive map, and extract route summary information such as total distance and estimated travel duration.
- To determine key route points, specifically the route start, the midpoint derived from the route polyline, and the destination.
- To calculate estimated arrival times for those points by combining departure time with route duration.
- To fetch weather forecast data for each route point and match the route timeline to the closest forecast timestamps.
- To generate contextual weather-risk warnings for conditions such as rain, thunderstorm, low visibility, and strong wind.

- To implement the full system in a frontend-only architecture suitable for educational deployment and mini-project demonstration.

Non-Functional Requirements

- The interface should remain responsive on desktop and mobile screens.
- The application should remain understandable without page reloads or multiple windows.
- The browser should provide clear feedback during route calculation and weather retrieval.
- The design should remain lightweight and easy to host because there is no backend.

3.4 Feasibility Analysis

Technical feasibility is high because all required tasks are available through documented APIs and JavaScript libraries. Browser-side asynchronous requests are sufficient for autocomplete, geocoding, forecast retrieval, and user-interface updates.

Operational feasibility is also high because the interaction model is simple. The user enters route information and departure time, then presses a single planning button. Results are progressively displayed without page reload.

Economic feasibility is favorable because the design relies on open or free-tier services and does not require a dedicated application server or database for prototype use.

3.5 Existing Fragmented Workflow

In the conventional fragmented workflow, a user computes a route in one application and checks weather in another. This means that no system automatically explains what the weather will be when the user reaches a route segment. Instead, the user must infer that manually. Such inference can be especially unreliable when the departure is planned for later in the day, when forecast panels are organized into discrete time blocks, or when different parts of the route traverse different localities.

From a systems-design perspective, the fragmented approach fails at integration rather than at data availability. Route data exist, and weather data exist, but they are not synchronized. The present application addresses exactly that integration gap.

CHAPTER 4

ROAD MAPPING AND PATH COMPUTATION ALGORITHM

4.1 Algorithm Description

The road mapping and path computation algorithm begins by reading the source and destination text entered by the user. If those fields already contain coordinates stored from an autocomplete selection, the application reuses them. Otherwise, it calls a fallback geocoding routine.

Once coordinates are available, the application instantiates a Leaflet Routing Machine control using the two route endpoints. When routing completes, the system reads the route summary and the ordered coordinate array returned in the route object.

The midpoint is derived as the coordinate located at the mathematical center of that array. This is a route-based midpoint and therefore differs from a straight-line geographical midpoint between source and destination.

4.2 Technologies Used

This algorithm depends primarily on OpenStreetMap, Nominatim, Leaflet, Leaflet Routing Machine, the OSRM-backed route service, and browser-side JavaScript. OpenStreetMap provides the foundational road-network and map context, while Nominatim converts place names into latitude–longitude coordinates and supports reverse geocoding for route-derived points.

Leaflet renders the interactive full-screen map, and Leaflet Routing Machine acts as the route-request and route-render orchestration layer. The application uses HTML to define the map container and input elements, CSS to support the interface layout, and JavaScript to coordinate geocoding, route calculation, and route-summary updates.

Data source / technology	Role in the system	Evidence from provided material
OpenStreetMap	Base road-network and map context for routing-related services	Named in project overview and used through tiles/geocoding/routing stack
Nominatim	Autocomplete search, geocoding, and reverse geocoding	Functions `geocode()` and midpoint reverse lookup in script.js
Leaflet	Interactive full-screen map rendering	Map element `#map` and `L.map()` initialization
Leaflet Routing Machine	Route request, route drawing, and route event handling	`L.Routing.control(...)` in script.js
OSRM-backed route service	Road path, route distance, and duration	Described in project overview as route-calculation engine
OpenWeather Forecast API	5-day forecast data in 3-hour chunks	`fetchForecast()` uses OpenWeather forecast endpoint
HTML / CSS / JavaScript	Structure, styling, and client-side logic	Uploaded `index.html`, `styles.css`, and `script.js`

Table 2. External services used by the application

4.3 Implementation Details

The code supports a current-location button for the start field through the browser geolocation API. When the user presses the button, the app attempts to capture the user's current coordinates, reverse-geocode them into a readable label, and populate the source field. Input validation is also present: the main button checks that both source and destination text are available before continuing.

After routing succeeds, the application formats and displays total distance in kilometers and estimated time in minutes. These values are shown together with short labels for source and destination. The route summary serves two purposes: it provides the user with standard navigation information and also acts as the basis for later route–weather interpretation because the duration value is required for timeline generation.

In stepwise form, the algorithm reads source and destination text from the interface, resolves the inputs into latitude–longitude coordinates using stored autocomplete values or fallback

geocoding, requests a drivable route between the two coordinates using Leaflet Routing Machine, receives the route polyline together with total distance and travel duration, computes the midpoint coordinate, and updates the map and route summary section.

4.4 Pseudocode and Architecture

- Read source and destination text from the UI.
- Resolve the inputs into latitude–longitude coordinates using stored autocomplete values or fallback geocoding.
- If either point cannot be resolved, stop the workflow and display an error message.
- Request a drivable route between the two coordinates using Leaflet Routing Machine.
- Receive the route polyline, total distance, and total travel duration.
- Compute the midpoint index as the central element of the route coordinate array and extract the midpoint coordinate.
- Display the route on the map and update the route summary section.

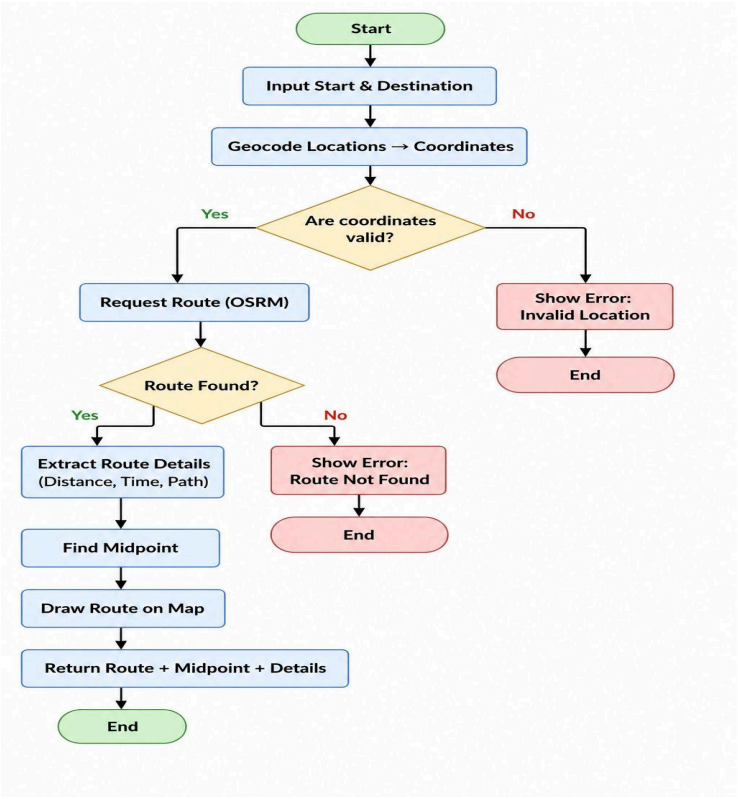

Fig 1

CHAPTER 5

WEATHER FORECAST EXTRACTION AND API INTEGRATION ALGORITHM

5.1 Algorithm Description

The weather forecast extraction and API integration algorithm prepares a list of coordinates corresponding to the start, midpoint, and destination. For each route point, the application constructs an OpenWeather forecast request URL using latitude, longitude, the API key, and metric units.

Instead of waiting for each forecast sequentially, the weather workflow launches all three requests concurrently using asynchronous JavaScript promises. This reduces the total waiting time perceived by the user and is a good example of asynchronous orchestration in browser JavaScript.

After each response is validated, the forecast list is extracted from the returned JSON object and stored for later time-based selection. At this stage the system has obtained the raw forecast arrays needed for the route-aware forecast-matching process.

5.2 Technologies Used

This algorithm is centered on the OpenWeather Forecast API together with JavaScript-based asynchronous requests. OpenWeather serves as the meteorological data source and provides the 5-day future forecast product returned in 3-hour chunks. Forecast entries contain timestamps and weather variables including temperature, humidity, wind speed, icon codes, and descriptive labels.

The wider frontend technology stack also supports this algorithm. HTML provides the document structure and content placeholders for the weather display, CSS supports the card-based visual presentation, and JavaScript performs the forecast requests, response handling, and later integration with the route timeline.

Layer	Technology	Implementation role
Document structure	HTML	Defines map container, inputs, summary zone, warnings, and weather cards
Presentation	CSS	Creates glassmorphism panel, responsive layout, and alert styling
Client logic	JavaScript	Coordinates API requests, timing logic, DOM updates, and warnings
Map rendering	Leaflet	Displays the interactive full-screen map
Routing control	Leaflet Routing Machine	Requests routes and exposes route events and geometry
Route service	OSRM-backed service	Supplies route polyline, distance, and duration
Geocoding	Nominatim	Place search, autocomplete, and reverse geocoding
Weather service	OpenWeather Forecast API	Supplies time-indexed forecast records

Table 4. Technical stack and implementation role

5.3 Implementation Details

Each weather card contains a location label, an arrival-time label, and a content region into which weather details are injected. The render function populates the card with forecast time, icon, rounded temperature, textual description, wind speed, and humidity. This card-based presentation is compact enough for a mini-project interface while still exposing the forecast details needed for travel judgement.

In stepwise form, the algorithm prepares the three route-point coordinates, constructs an OpenWeather request for each coordinate, fetches all forecast responses concurrently, validates the responses, extracts the forecast lists from the JSON objects, and stores the raw forecast arrays for later route-aware matching.

5.4 Pseudocode And Architecture

- Prepare a list of coordinates corresponding to start, midpoint, and destination.
- For each route point, construct an OpenWeather forecast request URL with latitude, longitude, API key, and units.
- Fetch all forecast responses concurrently using asynchronous JavaScript promises.
- Validate each response and extract the forecast list from the JSON object.
- Store the raw forecast arrays for later time-based selection.

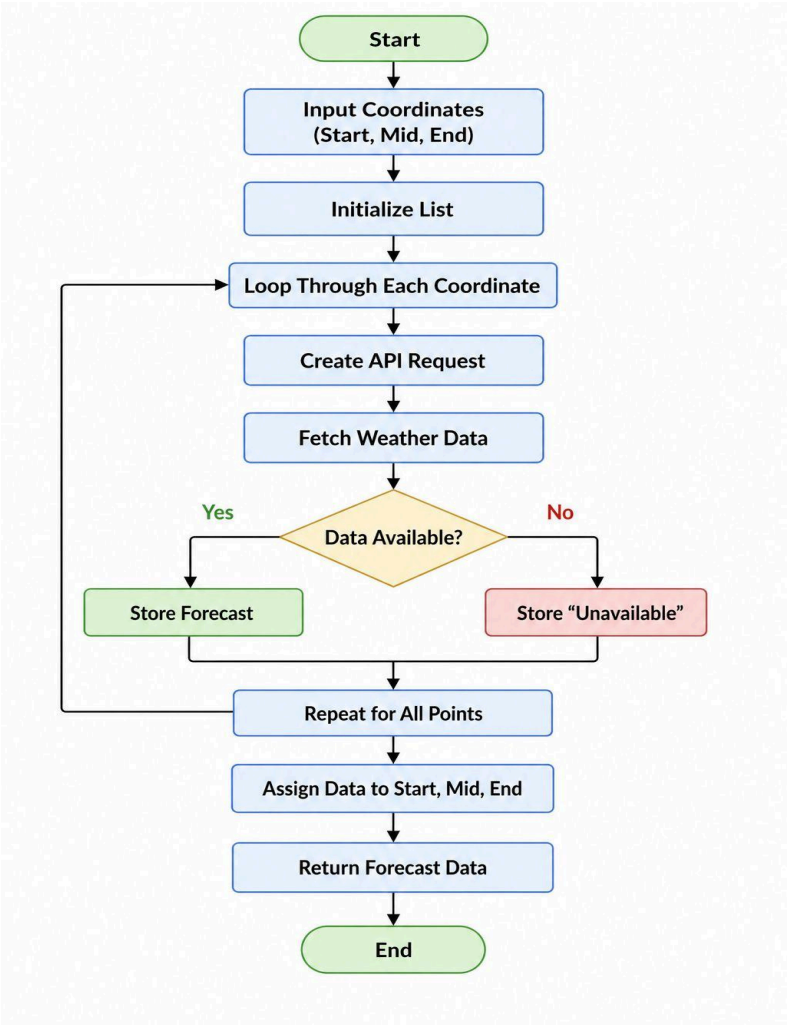

Fig 2

CHAPTER 6

ROUTE–WEATHER SYNCHRONISATION AND TIMELINE GENERATION ALGORITHM

6.1 Algorithm Description

The route–weather synchronization and timeline generation algorithm is the core timing logic of the application. The route engine provides travel duration in seconds, but the weather API provides multiple forecast records at discrete timestamps.

The application bridges these two data models by constructing three target times: the departure time itself, the midpoint time computed as departure plus half the route duration, and the destination time computed as departure plus the full route duration. Those times are then used as anchors against which forecast timestamps are compared.

The helper function `getClosestForecast()` implements this nearest-time selection. It first converts the target timestamp from milliseconds to seconds to match the units returned by the OpenWeather `dt` field. It then reduces the forecast list by repeatedly comparing the absolute difference between the target time and each candidate timestamp. The record with the smallest absolute difference is returned as the best available forecast for that route point.

Route point	Target time	Computation rule	Forecast selection
Start	Departure time	t0 = departure time	Nearest forecast to t0
Midpoint	Half of route duration	t1 = departure + totalDuration/2	Nearest forecast to t1
Destination	Full route duration	t2 = departure + totalDuration	Nearest forecast to t2

Table 3. Route-point and forecast-time matching concept

6.2 Technologies Used

This algorithm depends on the combined use of the routing engine and the OpenWeather forecast service. Leaflet Routing Machine and the OSRM-backed route service supply the total route duration, while OpenWeather supplies the time-indexed forecast records. JavaScript provides the arithmetic and comparison logic needed to compute target times and identify the nearest forecast record.

The route summary module also contributes to this algorithm because the total distance and travel duration displayed to the user are derived from the same route object that is later used for timeline generation.

6.3 Implementation Details

Read the user-selected departure time; if absent, use the current time. Use the total route duration returned by the routing engine to compute target timestamps for start, midpoint, and destination. Convert each target time into the same unit system used by the forecast API. For each route point, compare all forecast timestamps with the target time, and select the forecast record having the smallest absolute time difference.

This method is computationally simple and appropriate for the moderate list size returned by a 5-day 3-hour forecast API. In practical terms, it allows the application to show the forecast that is closest to the time at which the traveller is expected to reach each important point of the journey.

6.4 Pseudocode And Architecture

- Read the user-selected departure time; if absent, use the current time.
- Use the total route duration returned by the routing engine to compute target timestamps for start, midpoint, and destination.
- Convert each target time into the same unit system used by the forecast API.
- For each route point, compare all forecast timestamps with the target time.
- Select the forecast record having the smallest absolute time difference.
- Return the matched start, midpoint, and destination forecast records.

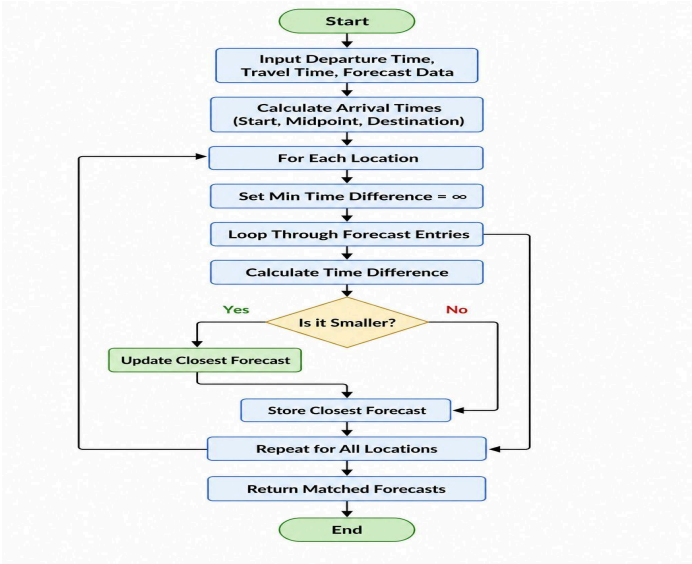

Fig 3

CHAPTER 7

WARNING GENERATION ALGORITHM

7.1 Algorithm Description

The warning generation algorithm operates after forecast records have been matched to the start, midpoint, and destination. It loops through the selected weather objects and extracts the condition code, main condition label, and wind speed.

If the code falls in the 2xx range, a thunderstorm warning is added. If the code falls in the 5xx range or the main condition text equals Rain, a rain warning is added. If the code falls in the 7xx range, a reduced-visibility warning is added. If wind speed exceeds 10 metres per second, a strong-wind warning is added.

Warnings are stored in a JavaScript `Set`, which ensures that duplicate messages are not repeated across route points.

7.2 Technologies Used

This algorithm depends on the OpenWeather forecast fields returned for each matched route point and on JavaScript condition-checking logic. The weather IDs, descriptive weather labels, and wind-speed values provide the raw input from which human-readable hazard advice is generated.

The warning module also depends on the frontend presentation layer. HTML provides the warning region in the interface, CSS supports the styled alert appearance, and JavaScript appends the final warning messages into the warnings section.

7.3 Implementation Details

The warning module transforms meteorological readings into direct advice. It does not simply print raw weather codes. Instead, it converts those into human-readable hazard alerts such as thunderstorm conditions, slippery roads due to rain, reduced visibility, or strong wind. This is important from a human–computer interaction perspective because users travel with decisions, not with numeric codes.

In stepwise form, the algorithm inspects the selected weather record at each route point, checks for thunderstorm, rain, visibility-related, and strong-wind conditions, adds the corresponding

human-readable warnings into a unique collection, and renders those warnings as styled alert cards in the interface.

7.4 Pseudocode

- Inspect the selected weather record at each route point.
- Check for thunderstorm condition codes in the 2xx range.
- Check for rain condition codes in the 5xx range or a main condition label of Rain.
- Check for atmospheric visibility-related codes in the 7xx range.
- Check whether wind speed exceeds the chosen threshold of 10 m/s.
- Add appropriate human-readable warning messages into a unique collection.
- Render those warnings as styled alert cards in the interface.

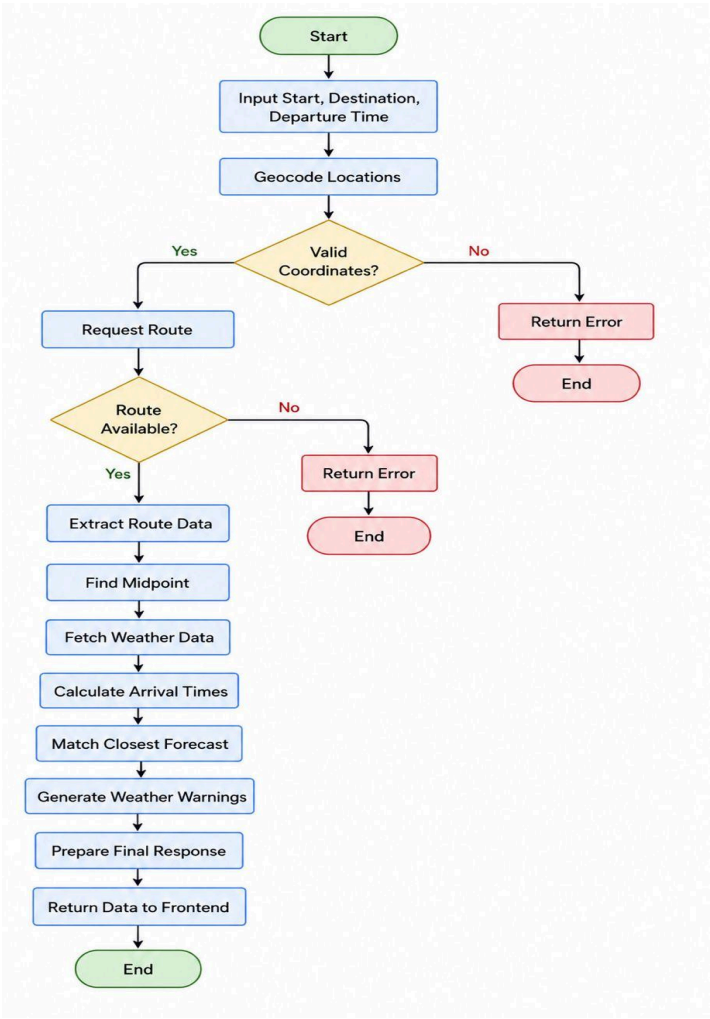

Fig 4

CHAPTER 8

FRONTEND ORCHESTRATION ALGORITHM

8.1 Algorithm Description

The frontend orchestration algorithm governs the overall event-driven flow of the application. The main planning button click listener performs validation, sets the departure time, shows loading states, clears previous warnings and weather cards, resolves coordinates, and triggers route calculation.

The `routesfound` event then updates the summary, adds a midpoint marker, computes times, and calls the weather workflow. Once weather retrieval and matching finish, individual card containers are updated through `renderWeather()`, while warning banners are appended into the warnings section. The button state is then restored.

For clarity, the technical workflow can be summarized as follows. Input text is converted to coordinates. Coordinates are converted to a route. The route is converted to route points and route duration. Route duration is converted to target times. Target times are compared with forecast timestamps. Matched forecasts are converted into visual cards and warnings.

8.2 Technologies Used

This algorithm relies on the coordinated use of HTML, CSS, and JavaScript across the user interface. The interface uses a full-screen map with an overlay panel rather than a traditional multi-page layout. This design allows the map to remain constantly visible while all interaction and result interpretation occur within a single visual frame.

JavaScript event listeners drive autocomplete, route generation, forecast retrieval, warning generation, and interface updates. The app operates through event listeners rather than sequential server-side request chains. Input events drive autocomplete. Button clicks trigger route and weather workflows. Routing completion triggers midpoint, timeline, and weather orchestration.

8.3 Implementation Details

The title, subtitle, source and destination inputs, current-location button, departure-time picker, route summary cards, warning alerts, and weather cards are all grouped in a progressive hierarchy so that the user is not shown irrelevant sections until the route is successfully computed.

In stepwise form, the algorithm captures user input events and performs debounced autocomplete search, validates input fields and sets loading state when the action button is clicked, clears previous route summary, warning, and weather outputs, triggers route calculation and then the weather workflow, renders the route summary together with weather cards and warnings, and finally restores the button state.

8.4 Pseudocode And Architecture

- Capture user input events and perform debounced autocomplete search.
- On action button click, validate input fields and set loading state.
- Clear previous route summary, warning, and weather outputs.
- Trigger route calculation; on success, compute midpoint and timeline.
- Trigger forecast retrieval and nearest-time selection.
- Render route summary, weather cards, and warnings; then restore button state.

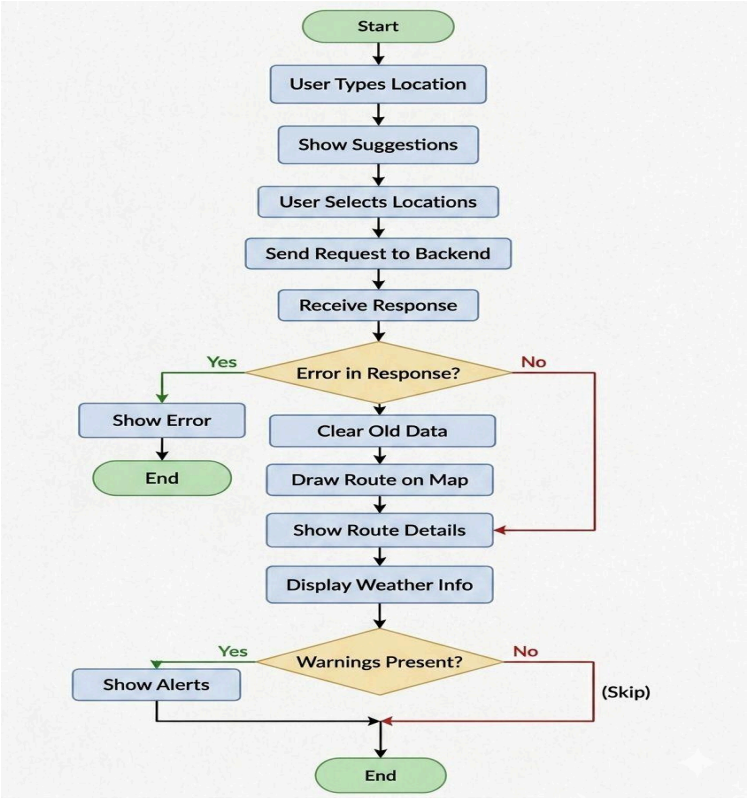

Fig 5.

CHAPTER 9

OUTPUT, DISCUSSION, ADVANTAGES AND LIMITATIONS

9.1 Output and Discussion

The key output of the application is a synchronized route-weather guidance view. The user receives not only a visible route, but also a route summary, a weather interpretation for three route points, and warnings relevant to those points. This makes the application more informative than a simple route viewer or city forecast viewer.

The uploaded paper is especially useful for interpreting the significance of this output. Ye et al. argue that navigation systems gain value when route information is fused with weather information, thereby enabling more informed route selection. Our application supports that idea, but in a focused form: instead of displaying a line graph of route temperature or multiple route alternatives, it gives a concise forecast snapshot at key route milestones and converts those into direct warnings.

The paper also discusses a limitation of fixed-time temperature representation and notes the importance of more dynamic updates. That observation is relevant here as well. Our app provides forecast guidance for the moment of planning, but does not continuously update the route or the forecast as the traveller progresses. It is therefore a predictive planning tool rather than a live route-tracking system.

With the addition of backend persistence and secured administration, the output of the system now includes not only predictive route-weather guidance for the traveller but also a recoverable historical record of requested journeys for administrative review. This materially strengthens the practical completeness of the project because it demonstrates that the application can move beyond momentary browser interaction into a modest but functional full-stack web workflow.

9.2 Advantages

- Frontend-only deployment keeps hosting simple and reduces infrastructure complexity.
- Open and free-tier services make the system suitable for an academic mini-project.
- Route-specific forecast matching provides richer decision support than destination-only weather display.

- Warning banners translate raw weather conditions into immediate travel guidance.
- The interface is visually cohesive and readable, combining spatial and card-based presentation.
- The implementation is educationally valuable because it demonstrates practical integration of multiple APIs in a browser environment.

Backend logging creates a reusable route-history repository that can support later review, debugging, or demonstration of usage patterns.

CAPTCHA-based verification and bcrypt-based password storage improve administrative security without complicating the end-user route-planning experience.

9.3 Limitations

- Nominatim is effective for towns and cities but less reliable than commercial services for highly specific places and local businesses.
- The OpenWeather 5-day product returns forecasts in 3-hour blocks, so the selected forecast is only an approximation of exact arrival-time weather.
- Free APIs can impose rate limits, which may affect scalability under heavy simultaneous use.
- The midpoint is derived from the route coordinate array and may not exactly represent the temporal halfway point of a journey on a highly curved route.
- The system currently works with a single route and three major route points rather than multiple waypoints or alternative route comparisons.
- The application does not continuously recompute its guidance in response to live movement, changing traffic, or newly updated forecasts.

Because persistence is file-based rather than database-server-based, the present backend design is well suited to prototype or mini-project deployment but would need further scaling controls for heavy multi-user production use.

Administrative history storage is intentionally limited to requested-route records and user credentials; it does not yet implement richer analytics, role-based authorization, or enterprise-grade database normalization.

CHAPTER 10

CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

The present report demonstrates that the final web app can be understood as a coherent, technically grounded response to a recognized gap in navigation systems: the absence of route-aware weather guidance. By combining geocoding, route computation, midpoint labeling, route timeline creation, forecast retrieval, nearest-time selection, and hazard generation, the application transforms separate route and weather services into a single travel-planning workflow.

The uploaded paper by Ye et al. strengthens the report by providing literature support for the broader concept of weather-aware route decision support. At the same time, the essence of the present system remains distinct and has been preserved throughout this revised report. The web app we created is a frontend-only route-weather planner focused on start, midpoint, and destination forecast alignment and warning generation. It is not a full commercial navigation engine and does not claim to be one. Its value lies in demonstrating the practical integration of open geospatial and forecast services in a clear, educational, and user-facing design.

10.2 Future Scope

- Increase route-point density by sampling multiple intermediate coordinates instead of only a single midpoint.
- Add waypoint support so that user-defined stops can be included in forecast matching.
- Support alternative route comparison and compare associated weather risks.
- Introduce a backend layer for API-key protection, response caching, and more robust orchestration.
- Use live GPS tracking to update user position and refresh route-related forecast guidance during the journey.
- Expand weather interpretation beyond rain and wind to include visibility, precipitation intensity, pressure change, or comfort metrics.
- Explore more dynamic route adaptation models inspired by the literature on static versus dynamic route-finding.

Replace file-based persistence with a relational or document-oriented database when the project evolves toward multi-user deployment, richer audit trails, or analytical dashboards of stored route requests.

CHAPTER 11

REFERENCES

1. [1] A. Anwar, “Google Maps weather along route: Weather-aware planner,” Afi Labs Blog, Jun. 2, 2025. [Online]. Available: https://blog.afi.io/blog/google-maps-weather-along-route-weather-aware-planner/. [Accessed: Aug. 22, 2026].

2. [2] E. B. Setiawan, G. P. Husein, and A. Setiyadi, “Travel Route Recommendation System Based on Weather Prediction and Geolocation Technology,” in 2023 International Conference on Informatics Engineering, Science & Technology (INCITEST), 2023, doi: 10.1109/INCITEST59455.2023.10395920.

3. [3] X. Ye, S. Li, S. Das, and J. Du, “Enhancing routes selection with real-time weather data integration in spatial decision support systems,” Spatial Information Research, Nov. 2023, doi: 10.1007/s41324-023-00564-8.

CHAPTER 12

PUBLICATIONS

Predictive Weather Guidance with Route Engine

Nimish M Biju, Riya Roy, Nihal Mohamed Shafy
Nandhakrishna VS, Pavithra R Raj
nimishmbiju@gmail.com
Dept of Computer Science, University College of Engineering, Kariyavattom
Guide:Anupama Muraleedharan

Abstract

This work presents Predictive Weather Guidance with Route Engine, a web-based application that combines navigation and meteorological forecasting in a single workflow. The system accepts a start location, destination, and departure time, converts place names into coordinates through Nominatim geocoding, computes a drivable route using Leaflet Routing Machine over an OSRM-backed service, and retrieves forecast records from the OpenWeather 5-day API. Instead of showing only current destination weather, it estimates the traveller's arrival time at the start, midpoint, and destination, then selects the forecast entry closest to each of those times. A warning module further interprets the matched weather conditions to highlight rain, thunderstorms, reduced visibility, and strong wind.

Traditional navigation systems calculate the fastest route between two points but frequently fail to account for dynamic environmental factors such as changing weather conditions over the duration of a journey. In practical travel, this limitation creates a fragmented workflow: the user consults one system for route guidance and another for weather, then manually attempts to combine the two. The present application addresses that gap by coupling route geometry with route timing and then coupling that timing with forecast data, thereby producing route-aware forecast guidance rather than destination-only weather display.

The frontend is implemented as a browser-based application using HTML, CSS, and JavaScript. OpenStreetMap and Nominatim support geocoding and reverse geocoding, Leaflet and Leaflet Routing Machine provide interactive map rendering and route computation, and OpenWeather supplies 5-day forecasts in 3-hour intervals. A synchronization stage computes expected arrival times for the start, midpoint, and destination and matches each of those times to the nearest forecast timestamp. The resulting forecasts are rendered into weather cards and interpreted through a warning module that highlights hazardous conditions such as rain, thunderstorm, fog, and strong wind.

In the documented full implementation, a lightweight Node.js and Express backend extends the route-prediction interface with route-history persistence, authenticated administrative access, CAPTCHA-based human verification, and bcrypt-based password security. The completed system therefore demonstrates not only a route-weather decision-support workflow but also a modest full-stack architecture in which predictive weather guidance, route-history storage, and controlled administration operate together in a single web application.

I. Introduction

Digital navigation systems are widely used for daily commuting, tourism, logistics, and inter-city travel. In their most basic form, route planners accept an origin and destination and return a path through the road network optimized for travel time or distance. Although this form of optimization is useful, it does not fully capture the realities of travel, because real journeys are also affected by weather, visibility, atmospheric hazards, and the time at which each route segment is reached.

Conventional travel planning is therefore fragmented. Travellers often consult a route application to determine the path and expected duration, then consult a separate weather application to inspect forecast conditions for one or more places. The user must mentally combine these outputs and infer whether the route is still suitable under forecasted conditions. This manual interpretation becomes especially difficult when departure is planned for later in the day, when forecast panels are organized into discrete time blocks, or when different

parts of the route pass through different localities with different atmospheric conditions.

The present project, Predictive Weather Guidance with Route Engine, addresses this gap by integrating route geometry, route timing, and forecast data in one workflow. Instead of merely showing a route together with current destination weather, the system attempts to estimate what weather will be encountered when the traveller reaches important route points. In this sense, the project is not simply a route planner with a weather panel and not merely a weather application with a map; it is a route-aware forecasting interface intended to improve pre-journey decision support.

The problem addressed by the system is therefore both practical and technical. Existing route planners usually optimize for distance or travel time without explicitly aligning forecasted weather with the user's planned travel timeline. Weather conditions vary along a route, and conditions at the origin, midpoint, and destination may differ considerably, especially for longer or geographically diverse journeys. A browser-based weather-aware planner is thus needed to provide route-specific forecast guidance and warning messages in a single interface using live geospatial and forecast services.

II. Literature Review

The literature shows that weather-aware travel planning is a recognized problem in digital navigation. Anwar describes a weather-aware route planner in which route guidance is enriched by live weather data, demonstrating the practical value of integrating navigation and forecast information. Setiawan, Husein, and Setiyadi also discuss travel-route recommendation using weather prediction and geolocation technology, showing that route planning can be improved when environmental conditions are treated as a relevant decision factor rather than as a separate afterthought.

The work by Ye, Li, Das, and Du is particularly relevant because it frames route selection with real-time weather data as a spatial decision-support problem. Their discussion makes clear that route details and weather information form separate streams that must be harmonized at the application layer. They also show that route guidance becomes more meaningful when meteorological conditions are considered in relation to actual travel rather than as a disconnected information layer. This conceptual framing strongly supports the central motivation of the present project.

The literature further explains that mainstream navigation tools usually anchor recommendations on metrics such as distance and projected travel time. These metrics are essential, but they are incomplete because they fail to account for rapidly changing atmospheric conditions. Prior work also discusses routing concepts such as shortest-path and safety-aware reasoning. Although the present application does not directly implement algorithms such as Dijkstra or A* in JavaScript, those discussions remain relevant because they clarify the broader class of problems that route engines solve and show that routing can be extended beyond pure shortest-distance logic toward safer travel decision support.

At the same time, the present project differs from more complex research-oriented systems. Rather than building a large-scale commercial optimization engine or a continuously adaptive routing platform, it uses open web services and a browser-based workflow to create a lightweight educational system. Its focus is on route-aware forecast matching, warning generation, and practical user-facing visualization. This positions the project as a modest but meaningful implementation of the broader literature on route-weather integration.

III. SYSTEM ANALYSIS AND PROPOSED DESIGN

a. Proposed System

The proposed system integrates user input capture, autocomplete, geocoding, route generation, midpoint extraction, reverse geocoding, timeline generation, forecast retrieval, nearest-time forecast matching, warning generation, and result rendering in a single-page browser application. The user provides the route and departure information once, and the application completes the remaining workflow automatically. The HTML, CSS, and JavaScript confirm this design. The HTML declares the structural placeholders for map, inputs, summary, warnings, and weather cards. The CSS defines the visual and responsive behavior. The JavaScript performs the operational orchestration that binds all system components together.

b. Overall Architecture

The application can be understood as a layered architecture moving from user input, through geocoding and routing, into route processing and timeline generation, and finally into forecast retrieval, warning generation, and presentation. This layered view is useful because it separates concerns: location interpretation, route geometry, temporal reasoning, environmental interpretation, and user communication are different technical problems even though they are executed in one browser workflow.

c. Functional and Non-Functional Requirements

Functional requirements include route generation, forecast retrieval, time-based forecast matching, and warning generation. Non-functional requirements include responsiveness, clarity of interaction, reasonable browser-side performance, lightweight deployment, and a visually coherent interface. The

project overview also emphasizes cost effectiveness, privacy-respecting behavior, and ease of hosting because no persistent backend is used.

d. Feasibility Analysis

Technical feasibility is high because all required tasks are available through documented APIs and JavaScript libraries. Browser-side asynchronous requests are sufficient for autocomplete, geocoding, forecast retrieval, and user-interface updates. Operational feasibility is also high because the interaction model is simple. The user enters route information and departure time, then presses a single planning button. Results are progressively displayed without page reload. Economic feasibility is favorable because the design relies on open or free-tier services and does not require a dedicated application server or database for prototype use.

e. Existing Fragmented Workflow

In the conventional fragmented workflow, a user computes a route in one application and checks weather in another. This means that no system automatically explains what the weather will be when the user reaches a route segment. Instead, the user must infer that manually. Such inference can be especially unreliable when the departure is planned for later in the day, when forecast panels are organized into discrete time blocks, or when different parts of the route traverse different localities. From a systems-design perspective, the fragmented approach fails at integration rather than at data availability. Route data exist, and weather data exist, but they are not synchronized. The present application addresses exactly that integration gap.

IV. ROAD MAPPING AND PATH COMPUTATION ALGORITHM

a. Algorithm Description

The road mapping and path computation algorithm begins by reading the source and destination text entered by the user. If those fields already contain coordinates stored from an autocomplete selection, the application reuses them. Otherwise, it calls a fallback geocoding routine. Once coordinates are available, the application instantiates a Leaflet Routing Machine control using the two route endpoints. When routing completes, the system reads the route summary and the ordered coordinate array returned in the route object. The midpoint is derived as the coordinate located at the mathematical center of that array. This is a route-based midpoint and therefore differs from a straight-line geographical midpoint between source and destination.

b. Technologies Used

This algorithm depends primarily on OpenStreetMap, Nominatim, Leaflet, Leaflet Routing Machine, the OSRM-backed route service, and browser-side JavaScript. OpenStreetMap provides the foundational road-network and map context, while Nominatim converts place names into latitude–longitude coordinates and supports reverse geocoding for route-derived points. Leaflet renders the interactive full-screen map, and Leaflet Routing Machine acts as the route-request and route-render orchestration layer. The application uses HTML to define the map container and input elements, CSS to support the interface layout, and JavaScript to coordinate geocoding, route calculation, and route-summary updates.

c. Implementation Details

The code supports a current-location button for the start field through the browser geolocation API. When the user presses the button, the app attempts to capture the user's current coordinates, reverse-geocode them into a readable label, and populate the source field. Input validation is also present: the main button checks that both source and destination text are available before continuing. After routing succeeds, the application formats and displays total distance in kilometers and estimated time in minutes. These values are shown together with short labels for source and destination. The route summary serves two purposes: it provides the user with standard navigation information and also acts as the basis for later route–weather interpretation because the duration value is required for timeline generation.

d.Pseudocode and Architecture

1. Read source and destination text from the UI.
2. Resolve inputs into lat-long using autocomplete or fallback geocoding.
3. If points fail to resolve, display error.
4. Request drivable route using Leaflet Routing Machine.
5. Receive polyline, distance, and travel duration.
6. Compute midpoint as central element of coordinate array.
7. Update map and summary section.

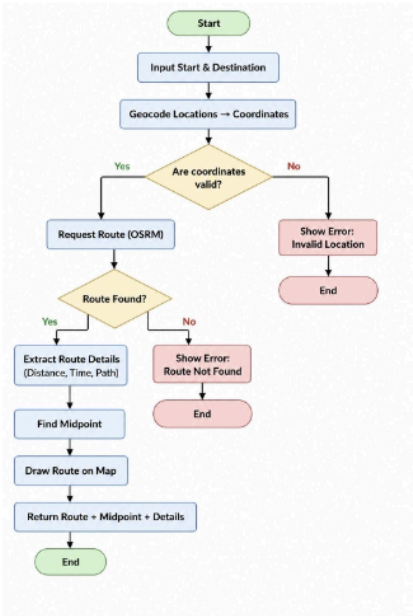

Fig 1: Road mapping and path computation flowchart

V. WEATHER FORECAST EXTRACTION

a. Algorithm Description

The weather forecast extraction and API integration algorithm prepares a list of coordinates corresponding to the start, midpoint, and destination. For each route point, the application constructs an OpenWeather forecast request URL using latitude, longitude, the API key, and metric units. Instead of waiting for each forecast sequentially, the weather workflow launches all three requests concurrently using asynchronous JavaScript promises. This reduces the total waiting time perceived by the user. After each response is validated, the forecast list is extracted from the returned JSON object and stored for later time-based selection.

b.Technologies Used

This algorithm is centered on the OpenWeather Forecast API together with JavaScript-based asynchronous requests. OpenWeather serves as the meteorological data source and provides the 5-day future forecast product returned in 3-hour chunks. Forecast entries contain timestamps and weather variables including temperature, humidity, wind speed, icon codes, and descriptive labels.

c. Implementation Details

Each weather card contains a location label, an arrival-time label, and a content region into which weather details are injected. The render function populates the card with forecast time, icon, rounded temperature, textual description, wind speed, and humidity. This card-based presentation is compact enough for a mini-project interface while still exposing the forecast details needed for travel judgement.

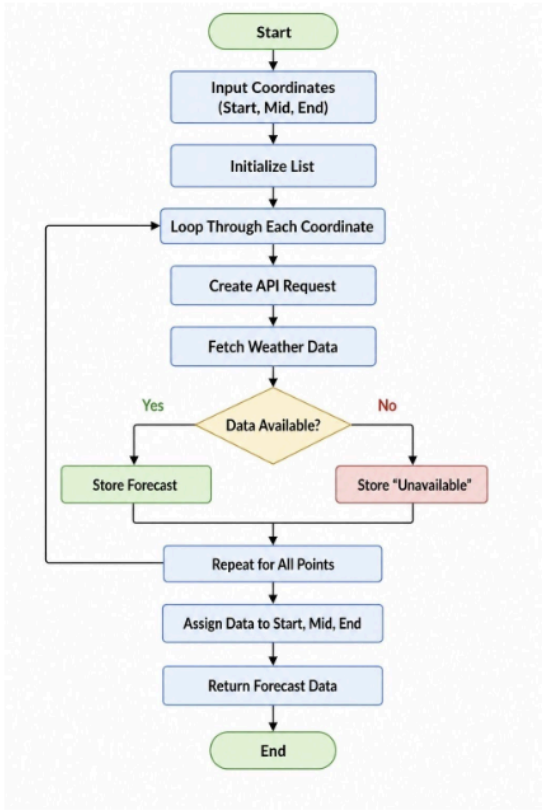

Fig 2: Weather forecast extraction flowchart

VI. ROUTE–WEATHER SYNCHRONISATION

a. Algorithm Description

The route–weather synchronization and timeline generation algorithm is the core timing logic of the application. The route engine provides travel duration in seconds, but the weather API provides multiple forecast records at discrete timestamps. The application bridges these two data models by constructing three target times: the departure time itself, the midpoint time computed as departure plus half the route duration, and the destination time computed as departure plus the full route duration. Those times are then used as anchors against which forecast timestamps are compared. The helper function getClosestForecast() implements this nearest-time selection.

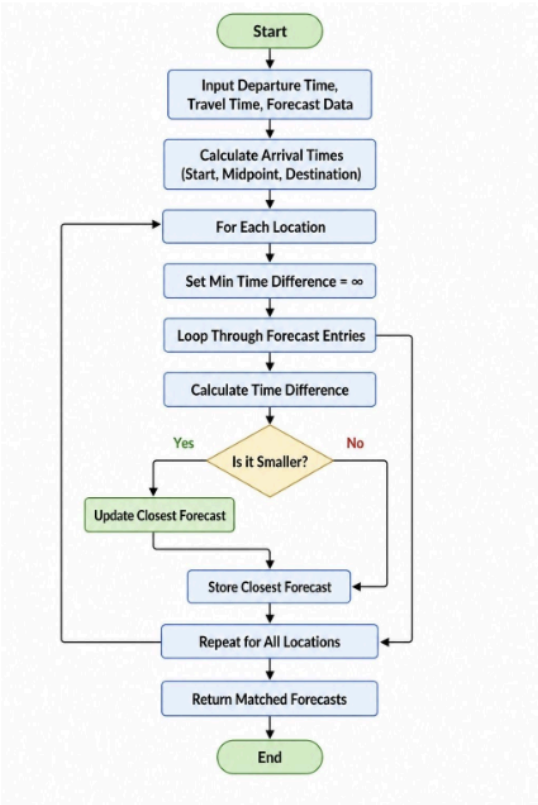

Fig 3: Route–weather synchronisation flowchart

VII. WARNING GENERATION ALGORITHM

The warning generation algorithm operates after forecast records have been matched. It loops through the selected weather objects and extracts the condition code, main condition label, and wind speed. If the code falls in the 2xx range, a thunderstorm warning is added. If the code falls in the 5xx range or the main condition text equals Rain, a rain warning is added. If the code falls in the 7xx range, a reduced-visibility warning is added. If wind speed exceeds 10 metres per second, a strong-wind warning is added. Warnings are stored in a JavaScript Set, which ensures that duplicate messages are not repeated across route points.

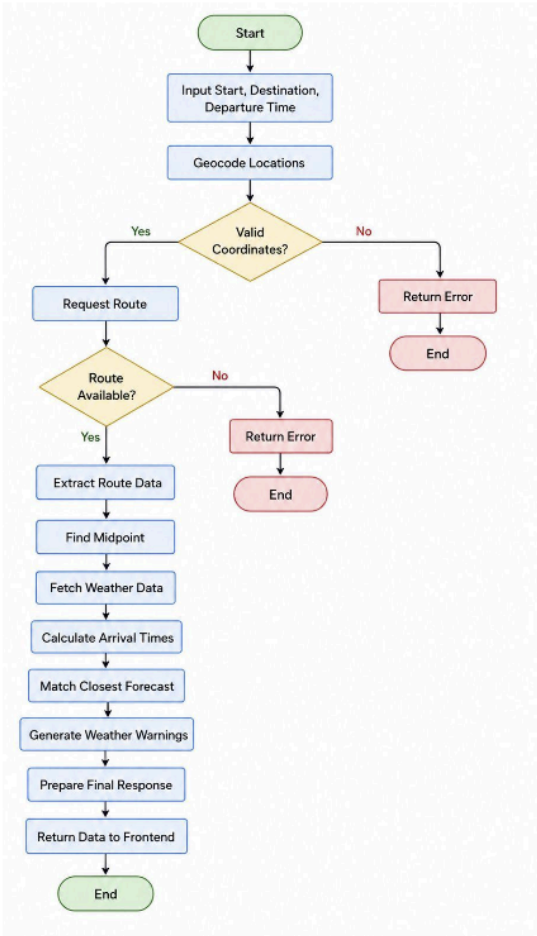

Fig 4: Warning generation logic flowchart

VIII. FRONTEND ORCHESTRATION ALGORITHM

The frontend orchestration algorithm governs the overall event-driven flow of the application. The main planning button click listener performs validation, sets the departure time, shows loading states, clears previous warnings and weather cards, resolves coordinates, and triggers route calculation. The routesfound event then updates the summary, adds a midpoint marker, computes times, and calls the weather workflow. Once weather retrieval and matching finish, individual card containers are updated through renderWeather(), while warning banners are appended into the warnings section.

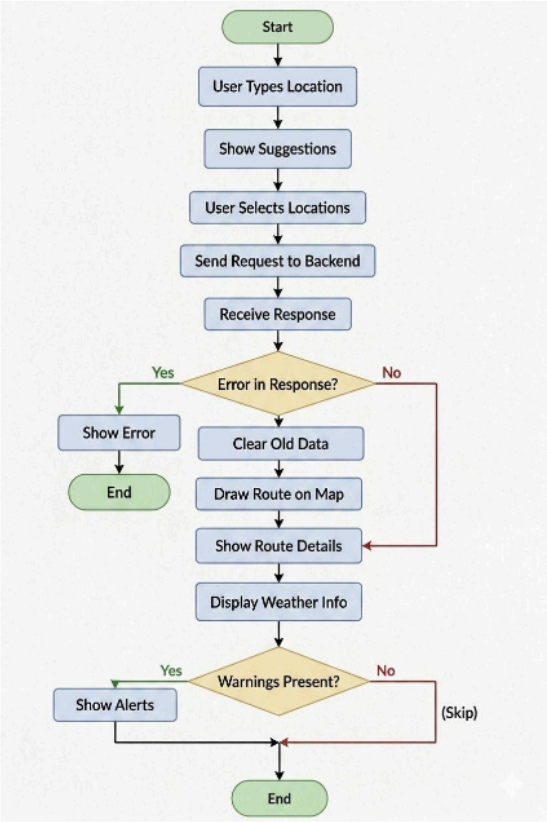

Fig 5: Frontend orchestration flowchart

IX. OUTPUT AND DISCUSSION

a. Output and Discussion

The key output of the application is a synchronized route-weather guidance view. The user receives not only a visible route, but also a route summary, a weather interpretation for three route points, and warnings relevant to those points. This makes the application more informative than a simple route viewer or city forecast viewer. Our application supports the idea that navigation systems gain value when route information is fused with weather information, thereby enabling more informed route selection. With the addition of backend persistence and secured administration, the output of the system now includes not only predictive route-weather guidance for the traveller but also a recoverable historical record of requested journeys for administrative review.

b. Advantages

• Frontend-only deployment keeps hosting simple.
• Route-specific forecast matching provides richer decision support.
• Warning banners translate raw conditions into immediate guidance.
• Visual glassmorphism interface.
• Backend logging for route history.
• Secure administrative access via CAPTCHA and bcrypt.

c. Limitations

• Nominatim is less reliable for specific business names than commercial services.
• Forecasts are in 3-hour blocks (approximation).
• Free API rate limits.
• Midpoint derivation is coordinate-based.
• No real-time GPS updating during the journey.

X. CONCLUSION AND FUTURE SCOPE

a. Conclusion

This report demonstrates that the final web app can be understood as a coherent, technically grounded response to the absence of route-aware weather guidance in conventional systems. By combining geocoding, route computation, midpoint labeling, route timeline creation, forecast retrieval, nearest-time selection, and hazard generation, the application transforms separate route and weather services into a single travel-planning workflow. Its value lies in demonstrating the practical integration of open geospatial and forecast services in a clear, educational, and user-facing design.

b. Future Scope

• Increase route-point density.
• Add user-defined waypoints.
• Support alternative route comparison.
• Use live GPS tracking for dynamic updates.
• Expand weather interpretation (comfort metrics).
• Replace file-based persistence with robust database servers.

References

[1] A. Anwar, "Google Maps weather along route: Weather-aware planner," Afi Labs Blog, Jun. 2, 2025.

[2] E. B. Setiawan, G. P. Husein, and A. Setiyadi, "Travel Route Recommendation System Based on Weather Prediction and Geolocation Technology," INCITEST, 2023.

[3] X. Ye, S. Li, S. Das, and J. Du, "Enhancing routes selection with real-time weather data integration in spatial decision support systems," Spatial Information Research, 2023.